

2.1 Indução: inferência a partir de casos e padrões

- Parte de observações/casos: identifica regularidades e propõe generalizações (hipóteses).
- Em computação: mineração de padrões, exemplos de entrada/saída, testes exploratórios e protótipos.
- Força: acelera descoberta e modelagem inicial; fraqueza: generalizações podem falhar fora do conjunto observado.
- Boa prática: explicitar suposições, coletar contraexemplos e refinar o modelo/algoritmo iterativamente.

2.2 Dedução: raciocínio formal e consequências lógicas

- Parte de premissas/regras: deriva conclusões por lógica (se... então...).
- Em computação: especificação (pré/pós-condições), invariantes, contratos, provas e tipos.
- Força: garante propriedades (correção/segurança) sob suposições explícitas; limitações: custo e esforço de formalização.
- Aplicações típicas: algoritmos críticos, sistemas distribuídos, segurança, concorrência e verificação de protocolos.

2.3 Complementaridade no ciclo: hipótese → teste → refinamento

- Indução ajuda a formular hipóteses/modelos; dedução ajuda a derivar implicações e checar consistência.
- Ciclo típico de desenvolvimento: observação → hipótese → implementação → testes → análise → refinamento.
- Padrões de engenharia: TDD, design iterativo, refatoração orientada por testes e especificações incrementais.
- Resultado esperado: convergir para soluções robustas com justificativas (empíricas e formais) do comportamento.

2.4 Implicações metodológicas: evidência, prova, validação e verificação

- Evidência empírica: experimentos, benchmarks, estudos de usuário, logs e análise estatística.
- Prova (formal): demonstra propriedades sob um modelo — depende das premissas e do escopo da especificação.
- Verificação vs. validação: “construí certo?” (verificação) vs. “construí a coisa certa?” (validação).
- Em pesquisa: transparência, reprodutibilidade, ameaças à validade e triangulação (múltiplas fontes de evidência).